Principles of Robot Autonomy I
Homework 3
Due Tuesday, October 27 (11:59pm PT)

Starter code for this homework has been made available online through GitHub. To get started, download the code by running `git clone https://github.com/PrinciplesofRobotAutonomy/AA274a-HW3-F26` in a terminal window. Install the dependencies with `pip install -r requirements.txt`. OpenCV is only used to detect the checkerboard corners in Problem 1; if it does not install on your machine, the notebook falls back to the corners saved in `data/corners.npz`.

You will submit your homework to Gradescope in two parts:

1. **HW3 Code**: upload `P1_extrinsics.py`, `P2_ransac.py`, `P3_icp.py`, and `P4_detection.py`. These are graded by an autograder (parts marked ⌨). Some tests run as soon as you submit, and the rest are released after the deadline, so passing the visible tests does not guarantee full credit. Before the deadline you will only see the results of the visible tests. You may resubmit as many times as you like; only your last submission counts. Do not rename the files or change the function signatures.
2. **HW3 Written**: a single pdf with your answers to the written questions and the requested plots (parts marked ✎). For short-answer questions, write your final answer on its own line starting with **Answer:**, followed by a brief justification where asked.

Your submission must be typeset in LaTeX.

You may discuss the problems with your classmates, but you must prepare your solutions on your own and list the names of the classmates you worked with. Using generative AI tools to substantially complete the assignment is not permitted, and any use beyond incidental use must be acknowledged in your submission. See the course outline for the full policy.

Introduction

This homework covers the perception side of the autonomy stack: estimating where a camera is from known features, making that estimate robust to bad correspondences, aligning 3D point clouds, and evaluating a learned object detector. Every problem has a notebook that runs and plots your code; all of your code goes in the four `.py` files.

Problem 1: Camera Extrinsics from a Checkerboard [21 Points]

Objective: Compute the position and orientation of a camera relative to a checkerboard of known size from a single image. This is how cameras localize against known markers, and it exercises projection between the world and the image plane. Your code goes in `P1_extrinsics.py`; the notebook is `p1_extrinsics.ipynb`.

Camera extrinsics. The rotation $\mathbf{R} \in SO(3)$ and translation $\mathbf{t} \in \mathbb{R}^3$ that map a point $\mathbf{P}$ in the world frame to the camera frame,

$$\mathbf{P}' = \mathbf{R}\mathbf{P} + \mathbf{t}, \tag{1}$$

are called the camera *extrinsics*. Note that $\mathbf{R}$ and $\mathbf{t}$ describe the world frame as seen from the camera. Here the world frame lies on the checkerboard, with the Z axis normal to the board. The squares are 0.0205 m wide and the board has 9×7 inner corners, which `cv2.findChessboardCorners` detects in pixel coordinates (Figure 1). The intrinsics matrix $\mathbf{K}$ is given.

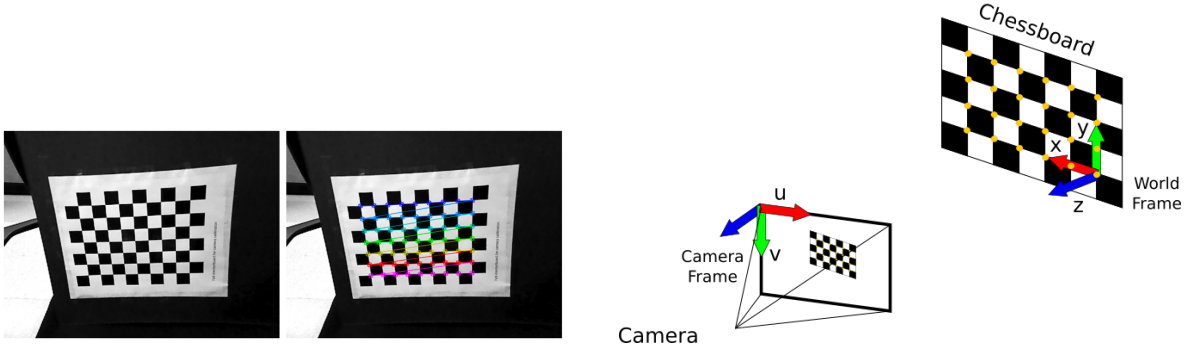


Figure 1: Corner (cross-junction) detection (left) and the checkerboard setup (right).

Homography. All board points have $Z = 0$, so the projection of a board point reduces to a 3×3 homography:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \propto \mathbf{K}[\mathbf{r}_0\ \mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}] \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} = \mathbf{K}[\mathbf{r}_0\ \mathbf{r}_1\ \mathbf{t}] \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}, \tag{2}$$

where $\mathbf{r}_i$ is the i-th column of $\mathbf{R}$. Each correspondence gives two linear equations in the entries of $\mathbf{H}$:

$$\mathbf{M}_i\mathbf{h} = \begin{bmatrix} -X_i & -Y_i & -1 & 0 & 0 & 0 & u_iX_i & u_iY_i & u_i \\ 0 & 0 & 0 & -X_i & -Y_i & -1 & v_iX_i & v_iY_i & v_i \end{bmatrix} \mathbf{h} = 0, \tag{3}$$

where $\mathbf{h} \in \mathbb{R}^9$ stacks the rows of $\mathbf{H}$. Stacking $\mathbf{M}_i$ for all corners gives $\mathbf{M}$, and $\mathbf{h}$ is the right singular vector of $\mathbf{M}$ with the smallest singular value (the direct linear transform, DLT).

Pose from the homography. From Eq. (2), $\mathbf{K}^{-1}\mathbf{H} = \lambda[\mathbf{r}_0\ \mathbf{r}_1\ \mathbf{t}]$ for some scale λ, which can be negative because $\mathbf{H}$ is only defined up to scale. Divide $\mathbf{K}^{-1}\mathbf{H}$ by the norm of its first column, choose the overall sign so that $t_z > 0$ (the board is in front of the camera), and set $\mathbf{r}_2 = \mathbf{r}_0 \times \mathbf{r}_1$. With noisy data $[\mathbf{r}_0\ \mathbf{r}_1\ \mathbf{r}_2]$ is not exactly a rotation, so replace it by the closest rotation matrix: if $[\mathbf{r}_0\ \mathbf{r}_1\ \mathbf{r}_2] = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, use $\mathbf{R} = \mathbf{U}\,\mathrm{diag}(1, 1, \det(\mathbf{U}\mathbf{V}^T))\,\mathbf{V}^T$.

(i) ⌨ (**10 Points**) Implement the following functions in `P1_extrinsics.py`, following the docstrings:

- `checkerboard_points`: world coordinates of the corners.
- `compute_homography`: the DLT estimate of $\mathbf{H}$.
- `pose_from_homography`: $\mathbf{R}$ and $\mathbf{t}$ as described above.
- `project_points`: pixel coordinates of world points.
- `reprojection_rmse`: root-mean-square pixel error.

Run the notebook up to and including "World to Camera Projection".

(ii) ✎ (**3 Points**) Report $\mathbf{R}$ and $\mathbf{t}$ (two significant figures), the reprojection RMSE, and the values of $\|\mathbf{r}_1\|$ and $\mathbf{r}_0 \cdot \mathbf{r}_1$ printed for $\mathbf{K}^{-1}\mathbf{H}$ after dividing by the norm of its first column.

(iii) ✎ (**3 Points**) Explain why the sign check on t_z and the projection onto the closest rotation are needed. For a perfect $\mathbf{K}$ and noise-free corners, what would $\|\mathbf{r}_1\|$ and $\mathbf{r}_0 \cdot \mathbf{r}_1$ be, and what do your values from (ii) suggest?

(iv) ✏️ (**3 Points**) Run the "Errors in the Intrinsics" cell, which recomputes the pose with both focal lengths scaled by 0.95 and 1.05. How do the components of $\mathbf{t}$ and the rotation change? Explain why t_z changes roughly in proportion to the focal length. Which scale gives the lowest reprojection error, and what does that suggest about the given $\mathbf{K}$? How would you estimate $\mathbf{K}$ more reliably?

(v) ✏️ (**2 Points**) Include `plots/p1_reprojection.png`, which shows the detected and reprojected corners and a secret point cloud projected onto the board.

Problem 2: Robust Estimation with RANSAC [12 Points]

Objective: Make the homography from Problem 1 robust to wrong correspondences. Feature matchers routinely return some matches that are simply wrong, and a single bad match can ruin a least-squares fit. Your code goes in `P2_ransac.py` (it uses your `compute_homography` from Problem 1); the notebook is `p2_ransac.ipynb`.

In the notebook, 30% of the detected checkerboard corners are replaced by random pixels. RANSAC (random sample consensus) repeatedly fits a model to a minimal random sample, counts how many correspondences agree with it, and keeps the best one:

1. Draw $s = 4$ correspondences at random (without replacement) and fit $\mathbf{H}$ to them.
2. Compute the reprojection residual of every correspondence under $\mathbf{H}$; the ones below a threshold are the inliers of this sample.
3. Repeat for a fixed number of iterations and keep the sample with the most inliers.
4. Refit $\mathbf{H}$ to all inliers of the best sample and recompute the inlier set under the refit $\mathbf{H}$.

If a fraction w of the correspondences are inliers, the number of iterations needed to draw at least one all-inlier sample with probability p is

$$N = \left\lceil \frac{\log(1-p)}{\log(1-w^s)} \right\rceil .$$

(i) ⌨️ (**6 Points**) Implement `homography_residuals`, `ransac_num_iterations`, and `ransac_homography` in `P2_ransac.py`. Use the provided random generator, `rng.choice(N, 4, replace=False)`, to draw samples. Run the notebook.

(ii) ✏️ (**2 Points**) Report N for $p = 0.99$ and $w = 0.7$ with $s = 4$ and with $s = 8$. In one sentence, explain why N grows so quickly with the sample size.

(iii) ✏️ (**4 Points**) Include `plots/p2_ransac.png`. Report the rotation error of the least-squares pose computed from all corrupted correspondences and of the RANSAC pose. Explain why least squares is so sensitive to a few bad correspondences. Why is the minimal sample 4 correspondences, and why are some samples drawn from a checkerboard useless for fitting a homography?

Problem 3: Iterative Closest Point (ICP) [19 Points]

Objective: Register a partial 3D scan of the Stanford Bunny to the full bunny model, as a robot would align a new lidar scan to a map. Your code goes in `P3_icp.py`; the notebook is `p3_icp.ipynb`. Everything uses NumPy and SciPy only. Use `scipy.spatial.cKDTree` for the nearest-neighbor search; computing all pairwise distances between the clouds does not fit in memory.

ICP alternates between finding correspondences and solving for the rigid transform that best aligns them. Given a current estimate $\mathbf{T}$ of the transform from the source to the target:

1. Transform the source points by $\mathbf{T}$ and find the nearest target point of each one.
2. Find the rigid transform $\Delta\mathbf{T}$ that minimizes $\sum_i \|\mathbf{R}\mathbf{a}_i + \mathbf{t} - \mathbf{b}_i\|^2$ over these pairs, and update $\mathbf{T} \leftarrow \Delta\mathbf{T}\,\mathbf{T}$.
3. Stop when the mean nearest-neighbor distance changes by less than a tolerance.

For corresponding points $\mathbf{A}, \mathbf{B}$ with centroids $\bar{\mathbf{a}}, \bar{\mathbf{b}}$, the optimal transform has a closed form: if $\sum_i (\mathbf{a}_i - \bar{\mathbf{a}})(\mathbf{b}_i - \bar{\mathbf{b}})^T = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, then $\mathbf{R} = \mathbf{V}\,\mathrm{diag}(1, 1, \det(\mathbf{V}\mathbf{U}^T))\,\mathbf{U}^T$ and $\mathbf{t} = \bar{\mathbf{b}} - \mathbf{R}\bar{\mathbf{a}}$. The determinant term prevents the solution from being a reflection.

ICP needs a reasonable initial guess; here it is provided (`T_init`), as it would come from a global method such as feature matching with RANSAC. A reference alignment `T_ref` is also provided, only to measure errors in the notebook.

(i) ⌨ (**9 Points**) Implement `nearest_neighbors`, `best_fit_transform`, and `icp` in `P3_icp.py`, including the optional outlier rejection: when `reject_dist` is given, only pairs closer than `reject_dist` are used to fit $\Delta\mathbf{T}$. Run the notebook.

(ii) ✎ (**4 Points**) Run the "Convergence Basin" cell, which starts ICP from the reference alignment rotated by 15°, 30°, 60°, 90°, and 120° about the vertical axis. Include the table it prints. Explain why ICP fails beyond some angle even though it reduces the mean nearest-neighbor distance at every iteration. Why does this make a good initial guess necessary?

(iii) ✎ (**4 Points**) Run the "Outlier Rejection" cell, in which the scan also contains points on the floor that have no match in the model. Report the rotation and point errors with and without `reject_dist = 0.005`. Explain why the floor points bias plain ICP. With rejection, the alignment is nearly perfect, yet the final mean nearest-neighbor distance is still about 1 cm; why? What goes wrong if `reject_dist` is chosen much too small or much too large?

(iv) ✎ (**2 Points**) Include `plots/p3_icp.png` and `plots/p3_clutter.png`.

Problem 4: Evaluating an Object Detector [18 Points]

Objective: Post-process and evaluate the output of a pretrained convolutional neural network for object detection. Your code goes in `P4_detection.py`; the notebook is `p4_detection.ipynb`.

The detections come from the FCOS (ResNet-50 FPN) detector in `torchvision`, pretrained on COCO, run on three images with a score threshold of 0.05 and *without* non-maximum suppression. They are saved in `data/detections.npz`, together with the indices the detector keeps after its own per-class NMS with an IoU threshold of 0.6, so parts (i) to (iii) do not need PyTorch. In part (iv) you run the detector yourself. Each detection is a box (x_1, y_1, x_2, y_2) in pixels, a class label, and a confidence score in $[0, 1]$.

Intersection over union (IoU) of two boxes is the area of their intersection divided by the area of their union. **Non-maximum suppression (NMS)** removes duplicate detections of the same object: repeatedly keep the highest-scoring remaining box and remove every remaining box whose IoU with it exceeds a threshold. Detectors apply NMS separately for each class.

Precision and recall. Given hand-labeled ground-truth boxes, detections are processed in decreasing score order, and each one is compared with the ground-truth box it overlaps most: it is a true positive if that IoU is ≥ 0.5 and that box has not been matched yet, and a false positive otherwise. Keeping only the k highest-scoring detections gives precision $= \mathrm{TP}/k$ and recall $= \mathrm{TP}/G$, where G is the number of ground-truth boxes. Sweeping k traces the precision-recall curve, and the **average precision (AP)** is the area under it after replacing each precision by the largest precision at any higher recall.

(i) 💻 (**11 Points**) Implement `box_iou`, `nms`, `batched_nms`, `match_detections`, `precision_recall`, and `average_precision` in `P4_detection.py`, following the docstrings. The notebook checks that your `batched_nms` reproduces the detector's own NMS exactly. Run the notebook.

(ii) ✏️ (**2 Points**) For the airplanes in the airport image, report the AP and the precision and recall at a score threshold of 0.5.

(iii) ✏️ (**2 Points**) Include `plots/p4_cars.png` and `plots/p4_pr.png`. Using the table and the precision-recall curve, explain what happens to the detections, the precision, and the recall if the score threshold is raised to 1 and if it is lowered to 0.01. Why is AP lower without NMS?

(iv) ✏️ (**3 Points**) Run the detector yourself. Install PyTorch with `pip install torch torchvision` (the CPU version is fine), or run the notebook in Google Colab and upload `P4_detection.py` and your photo next to it. Take a photo with several objects from the COCO classes (people, cars, bicycles, chairs, cups, bottles, laptops, ...), complete the cell in "Running the Detector Yourself", and include `plots/p4_own_image.png`, which shows the raw detections and the detections after your NMS. In two or three sentences: which objects were detected or missed, and why must the image be converted to floats in $[0, 1]$ and passed to the model as a list (a batch) of images?